Self-Paced Module: The Java Collections Framework

Java Programming · Grade 12 TVL · Cielito Zamora Senior High School · Week 14, October 19–23, 2026 · Continues the Exception Handling module

Name: _____
Section: _____ Date: _____

Day	Time	Topic	What you will do
Mon, Oct 19	2 hours	Introduction; List: ArrayList and LinkedList	Pages 1–2, Steps 1–2. Answer the **Monday strip** (page 7, bottom).
Tue, Oct 20	2 hours	Set: HashSet and TreeSet	Pages 2–3, Steps 3–4. Answer the **Tuesday strip** (page 7, middle).
Wed, Oct 21	2 hours	Map: HashMap and TreeMap	Pages 3–4, Steps 5–6. Answer the **Wednesday strip** (page 7, top).
Thu, Oct 22	3 hours	Iteration: for loops and Iterator	Pages 4–5, Steps 7–10. Answer the **Thursday strip** (page 8, bottom). Start your Performance Task.
Fri, Oct 23	3 hours	**PERFORMANCE TASK**	Page 6. Finish, present, and answer questions. Answer the **Friday strip** (page 8, top).

How to use this module: Work in order. Type every sample code and run it — do not just read it. Each day ends with a LIPLET, a short written quiz on **page 7 or 8**. Tear off **only that day's strip**, so you keep the rest of the module until Friday.

No internet needed. Everything you need is in this module and on your computer with Java (JDK) installed. Keep every class in its own file, all in one folder. Compile with `javac *.java`, run with `java Main`. Stuck on an error? See the **Error Guide on page 5**.

WHERE WE LEFT OFF — your Vehicle program from the Exception Handling module

This week continues the **same program**. Your folder should already have: `Vehicle.java` (with `getCapacity()`, `board()`, `quoteFare()`), `Jeepney.java`, `Tricycle.java`, `Bus.java`, `OverloadException.java`, and `InvalidDistanceException.java`. You will **replace** `Main.java` many times this week, so save a copy first as `Main_week13.txt`.

PART A — WHAT IS THE JAVA COLLECTIONS FRAMEWORK?

Definition: A **collection** is an object that holds a group of other objects. The **Java Collections Framework** is the set of ready-made **interfaces** (what a collection can do) and **classes** (working versions you can create with `new`) that Java gives you for storing, finding, and looping through groups of data. It lives in the `java.util` package.

Why not just use an array? Last week our fleet was `Vehicle[] fleet = new Vehicle[3];`. An array's size is **fixed** when you create it: a 4th jeepney simply does not fit. A collection **grows and shrinks** by itself, and comes with methods for adding, removing, searching, and sorting.

ANALOGY A terminal manages its data in three different ways. The **dispatcher's logbook** lists trips in order, and the same jeepney can appear many times: that is a **List**. The **LTO registration list** names each plate number only once, with no repeats allowed: that is a **Set**. The **key cabinet** has a hook labelled with each plate number, so you find a key instantly by its label: that is a **Map**.

Interface	Rule	Classes you will use	In our program
List	Keeps **order**; allows **duplicates**; each item has an **index** 0, 1, 2…	`ArrayList`, `LinkedList`	the fleet; the line of waiting passengers
Set	**No duplicates**; no index	`HashSet`, `TreeSet` (sorted)	registered plates; stops on a route
Map	Stores **key → value** pairs; each key appears once	`HashMap`, `TreeMap` (sorted keys)	plate → vehicle; driver → trips

Reading the angle brackets: `List<Vehicle>` means "a List that holds Vehicles." The type inside `< >` must be a class, so for numbers you write `Integer`, not `int`, and `Double`, not `double`. Java converts between them for you automatically.

MONDAY STEP 1 — ArrayList: a fleet that grows

STEP 1 Replace `Main.java` with this. Notice the **4th vehicle** — an array of 3 could never hold it.

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Vehicle> fleet = new ArrayList<>();     // starts empty, grows as needed
        fleet.add(new Jeepney("NXA 123", "Mang Ramon", "Monumento-Divisoria"));
        fleet.add(new Tricycle("TRC 456", "Kuya Ben", "Bagong Silang TODA"));
        fleet.add(new Bus("BUS 789", "Aling Rosa", 50));
        fleet.add(new Jeepney("NYB 321", "Mang Tonyo", "Sangandaan-Recto"));   // a 4th!

        System.out.println("Vehicles in fleet: " + fleet.size());
        System.out.println("First vehicle: " + fleet.get(0).getPlateNumber());

        fleet.remove(1);                             // the tricycle leaves for repair
        System.out.println("After removal: " + fleet.size());

        for (Vehicle v : fleet) {
            v.showInfo();
        }

        try {
            fleet.get(5).showInfo();                 // there is no index 5
        } catch (IndexOutOfBoundsException e) {
            System.out.println("No vehicle there: " + e.getMessage());
        }
    }
}
```

Output:

```
Vehicles in fleet: 4
First vehicle: NXA 123
After removal: 3
NXA 123 | Driver: Mang Ramon | Passengers: 0/20
BUS 789 | Driver: Aling Rosa | Passengers: 0/50
NYB 321 | Driver: Mang Tonyo | Passengers: 0/20
No vehicle there: Index 5 out of bounds for length 3
```

ArrayList method	What it does
`add(item)`	puts the item at the **end**; the list grows by one
`get(index)`	gives the item at that position (the first is index **0**)
`remove(index)`	takes out the item at that position; everything after it **moves up** one place
`size()`	how many items are in the list right now
`contains(item)`	`true` if the item is in the list

Look closely: after `remove(1)`, the bus moved from index 2 to index 1. And `get(5)` threw an `IndexOutOfBoundsException` — the same kind of error you learned to catch last week.

MONDAY STEP 2 — LinkedList: the line at the terminal

A **LinkedList** is also a List, but each item is linked to the one before and after it, like passengers holding hands in a line. Adding or removing at the **front** or **back** is very fast, which makes it perfect for a waiting line.

STEP 2 Replace `Main.java`. Senior citizens go to the **front** of the line (priority lane).

```
import java.util.LinkedList;

public class Main {
    public static void main(String[] args) {
        LinkedList<String> line = new LinkedList<>();   // passengers waiting at the terminal
        line.addLast("Ana");
        line.addLast("Ben");
        line.addLast("Carla");
        line.addFirst("Lolo Pedro");                    // senior citizen: priority lane

        System.out.println("Waiting: " + line);
        System.out.println("Next to board: " + line.getFirst());

        String boarding = line.removeFirst();
        System.out.println("Now boarding: " + boarding);
        System.out.println("Still waiting: " + line + " (" + line.size() + " people)");
    }
}
```

Output:

```
Waiting: [Lolo Pedro, Ana, Ben, Carla]
Next to board: Lolo Pedro
Now boarding: Lolo Pedro
Still waiting: [Ana, Ben, Carla] (3 people)
```

Why `LinkedList<String> line` and not `List<String> line`? Methods like `addFirst()` and `removeFirst()` belong to LinkedList, not to every List. If you need them, the variable's type must be `LinkedList`.

	ArrayList	LinkedList
Picture it as	a row of numbered seats in a bus	a line of people holding hands
Fast at	`get(index)`: jump straight to seat number 500	adding or removing at the **front** or **back**
Slow at	removing near the front: everyone behind must shift	`get(index)`: must walk the line one person at a time
Use it for	most lists: the fleet, a class list, a price list	queues and waiting lines

Check yourself: add `line.addLast("Lola Maria");` right after `addFirst`. Is she placed in the priority lane? (No — `addLast` puts her at the back. To give her priority, which method should you use instead?)

LIPLET · MONDAY Stop here and answer the **MONDAY strip (bottom of page 7)**, in pen. Tear it off and submit it.

TUESDAY PART B — SET: NO DUPLICATES ALLOWED

Definition: A **Set** is a collection that **never holds the same item twice**. If you try to add something already there, the set simply refuses and `add()` returns `false`. A Set has **no index**, so there is no `get(0)`.

ANALOGY The LTO will never register two vehicles with the same plate number. If someone tries to register *NXA 123* again, the clerk says “already registered” and nothing changes. A Set behaves exactly like that clerk.

STEP 3 Replace `Main.java`. Watch what `add()` prints the second time we try *NXA 123*.

```
import java.util.HashSet;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        Set<String> registered = new HashSet<>();      // plate numbers: no duplicates allowed

        System.out.println(registered.add("NXA 123"));    // true: added
        System.out.println(registered.add("TRC 456"));    // true: added
        System.out.println(registered.add("NXA 123"));    // false: already there!

        System.out.println("Registered plates: " + registered.size());
        System.out.println("Is BUS 789 registered? " + registered.contains("BUS 789"));

        registered.remove("TRC 456");
        System.out.println("After removing TRC 456: " + registered.size());
    }
}
```

Output:

```
true
true
false
Registered plates: 2
Is BUS 789 registered? false
After removing TRC 456: 1
```

TUESDAY STEP 4 — TreeSet: no duplicates, and SORTED

A jeepney passes the same stops more than once in a day. We want the list of **different** stops, in **alphabetical order**.

STEP 4 Replace `Main.java`. The same seven stops go into a HashSet **and** a TreeSet so you can compare them.

```
import java.util.HashSet;
import java.util.Set;
import java.util.TreeSet;

public class Main {
    public static void main(String[] args) {
        String[] passedBy = { "Monumento", "Recto", "5th Avenue", "Blumentritt",
                              "Divisoria", "Recto", "Monumento" };

        Set<String> hashStops = new HashSet<>();
        TreeSet<String> treeStops = new TreeSet<>();      // TreeSet keeps them SORTED
        for (String stop : passedBy) {
            hashStops.add(stop);
            treeStops.add(stop);
        }

        System.out.println("Stops passed: " + passedBy.length);
        System.out.println("Different stops: " + treeStops.size());
        System.out.println("TreeSet, sorted: " + treeStops);
        System.out.println("First alphabetically: " + treeStops.first());
        System.out.println("Last alphabetically: " + treeStops.last());
        System.out.println("HashSet: " + hashStops);
    }
}
```

Step 4 output:

```
Stops passed: 7
Different stops: 5
TreeSet, sorted: [5th Avenue, Blumentritt, Divisoria, Monumento, Recto]
First alphabetically: 5th Avenue
Last alphabetically: Recto
HashSet: [Monumento, 5th Avenue, Recto, Blumentritt, Divisoria]
```

Your HashSet line may be in a different order — that is correct. A HashSet makes **no promise** about order. Notice also that *5th Avenue* comes first in the TreeSet: text is sorted character by character, and digits come before letters.

	HashSet	TreeSet
Duplicates?	never	never
Order	**no promised order**	always **sorted** (A to Z, smallest to largest)
Speed	fastest for `add`, `remove`, `contains`	a little slower, because it keeps everything sorted
Extra methods	—	`first()`, `last()`
Use it for	checking "is this already here?": registered plates	a unique list that must be shown in order: stops, names

Check yourself: add `"Divisoria"` to the `passedBy` array one more time and run again. "Stops passed" becomes 8, but "Different stops" stays **5**. Can you explain why?

LIPLET · TUESDAY Stop here and answer the **TUESDAY strip (middle of page 7)**, in pen. Tear it off and submit it.

WEDNESDAY PART C — MAP: FIND ANYTHING BY ITS KEY

Definition: A **Map** stores **pairs**: a **key** and its **value**. You give the map a key, and it hands you the value instantly, without searching item by item. Each key can appear **only once**; putting a value under an existing key **replaces** the old value. A Map is part of the Collections Framework but is **not** a List or Set: it stores pairs, not single items.

ANALOGY The terminal's **key cabinet**: each hook has a label (the **key**: a plate number) and holds a jeepney key (the **value**). The dispatcher never searches every hook; she reads the label and takes the key. Two hooks can never have the same label.

Map method	What it does
`put(key, value)`	stores the pair; if the key already exists, its value is **replaced**
`get(key)`	gives the value for that key, or `null` if the key is not there
`containsKey(key)`	`true` if the key is in the map
`getOrDefault(key, backup)`	the value, or the **backup** value if the key is missing
`remove(key)`, `size()`	removes a pair; counts the pairs

STEP 5a A missing plate deserves a clear error, so we add one more **custom exception** from last week's lesson. Create `VehicleNotFoundException.java`:

```
public class VehicleNotFoundException extends Exception {     // CHECKED
    public VehicleNotFoundException(String message) {
        super(message);
    }
}
```

STEP 5b Replace `Main.java`. The map's key is the plate; its value is the whole Vehicle object.

```
import java.util.HashMap;
import java.util.Map;

public class Main {
    // look up a vehicle by plate; throws our custom exception if it is not there
    static Vehicle findVehicle(Map<String, Vehicle> byPlate, String plate)
            throws VehicleNotFoundException {
        if (!byPlate.containsKey(plate)) {
            throw new VehicleNotFoundException("No vehicle with plate " + plate);
        }
        return byPlate.get(plate);
    }

    public static void main(String[] args) {
        Map<String, Vehicle> byPlate = new HashMap<>();     // key: plate, value: the vehicle
        byPlate.put("NXA 123", new Jeepney("NXA 123", "Mang Ramon", "Monumento-Divisoria"));
        byPlate.put("TRC 456", new Tricycle("TRC 456", "Kuya Ben", "Bagong Silang TODA"));
        byPlate.put("BUS 789", new Bus("BUS 789", "Aling Rosa", 50));

        System.out.println("Vehicles on record: " + byPlate.size());
        byPlate.get("BUS 789").showInfo();                  // instant lookup by key
        System.out.println("Has ZZZ 000? " + byPlate.containsKey("ZZZ 000"));
        System.out.println("get(\"ZZZ 000\") gives: " + byPlate.get("ZZZ 000"));

        try {
            findVehicle(byPlate, "TRC 456").showInfo();
            findVehicle(byPlate, "ZZZ 000").showInfo();     // not found
        } catch (VehicleNotFoundException e) {
            System.out.println("Lookup failed: " + e.getMessage());
        }
    }
}
```

Output:

```
Vehicles on record: 3
BUS 789 | Driver: Aling Rosa | Passengers: 0/50
Has ZZZ 000? false
get("ZZZ 000") gives: null
TRC 456 | Driver: Kuya Ben | Passengers: 0/4
Lookup failed: No vehicle with plate ZZZ 000
```

Why a custom exception? `get()` on a missing key quietly returns `null`. If you then call `.showInfo()` on that `null`, the program crashes with a `NullPointerException`. `findVehicle()` checks with `containsKey()` first and throws a clear, **checked** exception instead, so whoever calls it is forced to plan for a missing plate. This is Week 13's lesson working inside this week's collections.

WEDNESDAY STEP 6 — TreeMap: counting trips, sorted by driver

The dispatcher counts how many trips each driver made today. A **TreeMap** keeps its keys **sorted**, so the report comes out in alphabetical order by name.

STEP 6 Replace `Main.java`. The values are counts, so the type is `Integer`, not `int`.

```
import java.util.Map;
import java.util.TreeMap;

public class Main {
    public static void main(String[] args) {
        Map<String, Integer> trips = new TreeMap<>();     // key: driver, value: trips today
        trips.put("Mang Ramon", 5);
        trips.put("Kuya Ben", 8);
        trips.put("Aling Rosa", 3);

        trips.put("Kuya Ben", trips.get("Kuya Ben") + 1); // same key: the value is REPLACED
        trips.put("Mang Tonyo", trips.getOrDefault("Mang Tonyo", 0) + 1);

        System.out.println("Drivers: " + trips.size());
        for (Map.Entry<String, Integer> entry : trips.entrySet()) {
            System.out.println(entry.getKey() + ": " + entry.getValue() + " trip(s)");
        }
    }
}
```

Output:

```
Drivers: 4
Aling Rosa: 3 trip(s)
Kuya Ben: 9 trip(s)
Mang Ramon: 5 trip(s)
Mang Tonyo: 1 trip(s)
```

Read it carefully: Kuya Ben now has **9**, not 8 — putting under the same key replaced his old count. Mang Tonyo was not in the map yet, so `getOrDefault` used the backup value **0**, then added 1. Each `Map.Entry` is one key–value pair; Thursday explains this loop fully.

	HashMap	TreeMap
Keys repeat?	never; a repeated key replaces its value	never; a repeated key replaces its value
Order of keys	**no promised order**	always **sorted** by key
Use it for	fast lookup: plate → vehicle	reports that must be in order: driver → trips

Choosing the right collection — ask these questions in order

Question	If YES, use…
1. Do I need to find things by a **label** (a key)?	**Map**: `HashMap`, or `TreeMap` if keys must be sorted
2. Must every item be **unique**?	**Set**: `HashSet`, or `TreeSet` if it must be sorted
3. Does **order** matter, or can items repeat?	**List**: `ArrayList`, or `LinkedList` for a waiting line

LIPLET · WEDNESDAY Stop here and answer the **WEDNESDAY strip (top of page 7)**, in pen. Tear it off and submit it.

THURSDAY PART D — ITERATION: GOING THROUGH A COLLECTION

Iteration means visiting every item in a collection, one at a time. Java gives you three ways. Each one is right for a different job.

Loop	Shape	Works on	Choose it when…
1. Index for	`for (int i = 0; i < list.size(); i++)`	Lists only (needs an index)	you need the position number
2. For-each	`for (Vehicle v : fleet)`	any List, Set, or Map view	you only **read** each item; the shortest
3. Iterator	`Iterator<Vehicle> it = fleet.iterator();`	any List or Set	you must **remove** items while looping

STEP 7 Loops 1 and 2. Replace `Main.java`:

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Vehicle> fleet = new ArrayList<>();
        fleet.add(new Jeepney("NXA 123", "Mang Ramon", "Monumento-Divisoria"));
        fleet.add(new Tricycle("TRC 456", "Kuya Ben", "Bagong Silang TODA"));
        fleet.add(new Bus("BUS 789", "Aling Rosa", 50));
        fleet.get(1).setPassengers(3);

        // LOOP 1: index for loop (lists only) - you get the position number
        for (int i = 0; i < fleet.size(); i++) {
            System.out.println((i + 1) + ". " + fleet.get(i).getPlateNumber());
        }

        // LOOP 2: for-each loop (any collection) - shortest, read-only
        double total = 0;
        for (Vehicle v : fleet) {
            total = total + v.quoteFare(5);
        }
        System.out.println("Total fare for 5 km on every vehicle: P" + total);
    }
}
```

Output: *(P15.00 + P70.00 + P27.50: each vehicle used its own `computeFare()`, polymorphism from the Four Pillars)*

```
1. NXA 123
2. TRC 456
3. BUS 789
Total fare for 5 km on every vehicle: P112.5
```

THURSDAY STEP 8 — Removing inside a for-each loop: the wrong way

End of shift: every **empty** vehicle goes off duty. The obvious idea is to remove it inside a for-each loop.

STEP 8 Keep the first five lines of `main` from Step 7 (the fleet and `setPassengers(3)`). Replace the two loops with this, and run it:

```
        // remove every EMPTY vehicle (end of shift) - the WRONG way
        for (Vehicle v : fleet) {
            if (v.getPassengers() == 0) {
                fleet.remove(v);
            }
        }
        System.out.println("Vehicles still on duty: " + fleet.size());
```

```
Exception in thread "main" java.util.ConcurrentModificationException
        at java.base/java.util.ArrayList$Itr.checkForComodification(ArrayList.java:1095)
        at java.base/java.util.ArrayList$Itr.next(ArrayList.java:1049)
        at Main.main(Main.java:13)
```

What happened: a for-each loop secretly uses an iterator to walk the list. When you changed the list behind its back with `fleet.remove(v)`, the iterator noticed the list had been **modified while it was looping** and stopped the program. Your line numbers may differ.

THURSDAY STEP 9 — Loop 3: the Iterator, the right way

```
Iterator<Vehicle> it = fleet.iterator();
while (it.hasNext()) {     // is there another?
    Vehicle v = it.next();    // move to it
    // ... use v ...
    it.remove();              // remove v, safely
}
```

An Iterator is an object that walks through a collection one step at a time. `hasNext()` asks "is there another item?", `next()` moves to it, and `remove()` deletes the item you just visited. Because the **iterator itself** does the removing, it never loses its place.

STEP 9 Replace the wrong loop from Step 8 with this. Add `import java.util.Iterator;` at the top.

```
        // LOOP 3: Iterator - the SAFE way to remove while looping
        Iterator<Vehicle> it = fleet.iterator();
        while (it.hasNext()) {
            Vehicle v = it.next();
            if (v.getPassengers() == 0) {
                it.remove();                    // remove through the iterator
                System.out.println("Off duty: " + v.getPlateNumber());
            }
        }
        System.out.println("Vehicles still on duty: " + fleet.size());
```

```
Off duty: NXA 123
Off duty: BUS 789
Vehicles still on duty: 1
```

THURSDAY STEP 10 — Looping through a Set and a Map

A Set has no index, so use for-each. A Map is not looped directly; you loop over one of its three **views**: `keySet()` (keys only), `values()` (values only), or `entrySet()` (key and value together).

STEP 10 Replace `Main.java`:

```
import java.util.Map;
import java.util.Set;
import java.util.TreeMap;
import java.util.TreeSet;

public class Main {
  public static void main(String[] args) {
    Set<String> stops = new TreeSet<>();
    stops.add("Recto");
    stops.add("Monumento");
    stops.add("Divisoria");
    for (String stop : stops) {
      System.out.println("Stop: " + stop);
    }

    Map<String, Integer> trips = new TreeMap<>();
    trips.put("Mang Ramon", 5);
    trips.put("Kuya Ben", 9);
    trips.put("Aling Rosa", 3);

    for (String driver : trips.keySet()) {
      System.out.print(driver + "  ");      // keys
    }
    System.out.println();

    int total = 0;
    for (int count : trips.values()) {   // values
      total = total + count;
    }
    System.out.println("Total trips: " + total);

    for (Map.Entry<String, Integer> e
          : trips.entrySet()) {          // both
      System.out.println(e.getKey() + " -> "
                         + e.getValue());
    }
  }
}
```

```
Stop: Divisoria
Stop: Monumento
Stop: Recto
Aling Rosa  Kuya Ben  Mang Ramon
Total trips: 17
Aling Rosa -> 3
Kuya Ben -> 9
Mang Ramon -> 5
```

LIPLET · THURSDAY Stop here and answer the **THURSDAY strip (bottom of page 8)**, in pen. Tear it off, submit it, then start the Performance Task on page 6.

ERROR GUIDE — no internet? Look up your error here

If you see…	It means…	Fix it by…
`cannot find symbol … symbol: class ArrayList`	A missing import.	Add the import, e.g. `import java.util.ArrayList;`, or use `import java.util.*;`
`unexpected type … required: reference found: int`	You wrote a primitive inside `< >`.	Use the class name: `Integer`, `Double`, `Boolean`.
`incompatible types: String cannot be converted to Vehicle`	You added the wrong type to a `List<Vehicle>`.	Add a Vehicle object, e.g. `new Jeepney(...)`.
`cannot find symbol … symbol: method get(int) … type Set<String>`	Sets have no index.	Loop with for-each, or use a List if you need positions.
`cannot find symbol … symbol: method first() … type Set<String>`	`first()` belongs to TreeSet, not every Set.	Declare the variable as `TreeSet<String>`.
`IndexOutOfBoundsException: Index 5 out of bounds for length 3`	No item at that index (runs, then crashes).	Keep the index below `size()`, or catch the exception.
`ConcurrentModificationException`	The collection changed while a for-each loop was going through it.	Remove with an Iterator: `it.remove()` (Step 9).
`NullPointerException … the return value of "java.util.Map.get(Object)" is null`	You used `get()` with a key that is not in the map.	Check with `containsKey()` first, or use `getOrDefault()`.

THU-FRI PART E — PERFORMANCE TASK: MANAGE YOUR DATA WITH COLLECTIONS

Open **your own** program from last week's Performance Task (your sari-sari store, pets, gadgets, school personnel, or shapes theme). Replace its arrays with collections, and add a List, a Set, and a Map that each do a real job. Start on Thursday after the Thursday strip; finish and present on Friday.

Requirements checklist — tick each box when done:

- ☐ A **List** of your objects with **at least 4**, from at least 2 subclasses
- ☐ A **Set** that refuses a duplicate — show `add()` returning `false`
- ☐ A **Map** for looking up an object by a key (an ID, name, or code)
- ☐ A **find** method that throws your own checked **NotFound** exception
- ☐ Loop 1: an **index for** loop that numbers the items
- ☐ Loop 2: a **for-each** loop that adds up or counts something
- ☐ Loop 3: an **Iterator** that safely **removes** some items
- ☐ A Map loop using `entrySet()`
- ☐ The user **types a key**; a missing key shows a friendly message, never a crash
- ☐ A comment on each part, e.g. `// SET: no duplicate IDs`

Example for one theme (sari-sari store). Do yours **your own way**.

```
Map<String, Integer> stock = new TreeMap<>();
stock.put("Softdrink", 24);
stock.put("Bread", 10);
stock.put("Rice (kg)", 50);

List<String> salesLog = new ArrayList<>();
Set<String> customers = new TreeSet<>();

salesLog.add("Ana bought 2 Bread");
customers.add("Ana");
salesLog.add("Ben bought 1 Softdrink");
customers.add("Ben");
salesLog.add("Ana bought 1 Softdrink");
customers.add("Ana");          // refused

System.out.println("Sales today: "
    + salesLog.size());                // 3
System.out.println("Different customers: "
    + customers);                      // [Ana, Ben]
```

Day	First half (about 1.5 hours)	Second half (about 1.5 hours)	Before you leave
Thursday	Steps 7 to 10 on pages 4–5; Thursday strip	Fill in the plan below; build your List and Set	Your List and Set compile and run
Friday	Build the Map, find method, and loops; a classmate tries to break it	Presentations and Q&A	Friday strip handed in

Testing tip: before you present, run your program at least four times: once normally, once adding a **duplicate** to your Set, once typing a key that **exists**, and once typing a key that **does not exist**. Every run must end with a friendly message, never an `Exception in thread "main"` report.

Plan before you type — fill this in, and show it to your teacher

	Class I will use	What it holds (type in < >)	Why this one, and not the others?
List			
Set			
Map		key: value:	
Iterator	Which items will my Iterator remove, and why?		

Starter template for the lookup part of your `Main.java`

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter ________ to find: ");
String key = sc.nextLine();
try {
    ________ found = find________(map, key);
    found.showInfo();
} catch (________NotFoundException e) {
    System.out.println("Sorry: " + e.getMessage());
} finally {
    System.out.println("Search finished.");
}
```

Self-check before you present — tick only what is honestly true:

I can explain, using my own code...	Yes	Not yet
why I chose a List, a Set, and a Map	☐	☐
Hash versus Tree in the classes I chose	☐	☐
which loop I used where, and why	☐	☐
why removing needs an Iterator	☐	☐

Friday: your 5-minute presentation, in this order

1. **Introduce** your theme and your List, Set, and Map. *(30 sec)*
2. **Run it:** show the numbered list and the totals. *(1 min)*
3. **Prove the rules live:** add a duplicate to your Set; look up a key that exists and one that does not; remove items with your Iterator. *(2 min)*
4. **Point to the code** for each of the three loops and say why you chose it. *(1 min)*
5. **Answer** questions. *(30 sec)*

Prepare for these questions:

- Why is your Set a Set and not a List?
- What would change if your HashMap became a TreeMap?
- What happens if you remove inside your for-each loop?
- Why does your Map use `Integer` and not `int`?
- Which loop could not work on your Set, and why?

Scoring Rubric (50 points)

Criterion	What earns full points	Points	Score
List	Holds at least 4 objects from at least 2 subclasses; uses add, get, size, and remove correctly.	7	
Set	Fits a real “must be unique” rule; a duplicate is shown being refused.	7	
Map + exception	Lookup by key works; a missing key throws the custom checked exception and is caught.	8	
Iteration	Index for, for-each, and Iterator with remove each used for the right job; a Map loop with entrySet.	10	
Program quality	Compiles and runs; no crash on bad input; parts labelled in comments.	8	
Presentation and Q&A	Proves each rule live; explains every choice of collection and loop with own code.	10	
	TOTAL	**50**	

LIPLET · FRIDAY After your presentation, answer the **FRIDAY strip (top of page 8)**, in pen, and hand in that sheet.

WEDNESDAY · OCT 21 **Map: HashMap and TreeMap**

Name: _____
Section: _____ Score: ____ / 15

Answer after Step 6 on page 4. This is the last strip on this sheet — hand in what is left.

3.1 Define (5 pts): What is a Map? Explain *key* and *value* using the plate-number map from Step 5.

3.2 Explain (5 pts): In Step 6, Kuya Ben ended with **9** trips, not two separate entries of 8 and 1. Why?

3.3 Connect (5 pts): Why does `findVehicle()` throw a `VehicleNotFoundException` instead of just returning what `get()` gives? Use the word *NullPointerException*.

✂ CUT ON TUESDAY · DETACH THE TUESDAY STRIP BELOW THIS LINE ✂

TUESDAY · OCT 20 **Set: HashSet and TreeSet**

Name: _____
Section: _____ Score: ____ / 20

Answer after Step 4 on page 3.

2.1 Define (5 pts): What is a Set? In Step 3, what did `add("NXA 123")` return the second time, and why?

2.2 Explain (5 pts): In Step 4, seven stops were passed but there were only five “different stops.” Why? And why was the TreeSet's order different from the HashSet's?

2.3 Choose (5 pts): Write the best class (ArrayList, LinkedList, HashSet, or TreeSet) for each.

a) all LRNs in school, no repeats, any order → _____

b) learners' names in a section, A to Z → _____

c) order of homework submitted, repeats OK → _____

d) the line at the school canteen → _____

2.4 Connect (5 pts): Name one real record that must never contain duplicates. Would you use a HashSet or a TreeSet for it? Why?

✂ CUT ON MONDAY · DETACH THE MONDAY STRIP BELOW THIS LINE ✂

MONDAY · OCT 19 **Introduction; List: ArrayList and LinkedList**

Name: _____
Section: _____ Score: ____ / 15

Answer after Step 2 on page 2.

1.1 Define (5 pts): What is the Java Collections Framework? Give one reason to use an ArrayList instead of an array.

1.2 Explain (5 pts): In Step 1, after `fleet.remove(1)`, the bus moved from index 2 to index 1. Why?

1.3 Connect (5 pts): Name one waiting line in your school or barangay. Why is a LinkedList a better fit for it than an ArrayList?

FRIDAY · OCT 23 Performance Task Exit Questions

Name: _____
Section: _____ Score: ____ / 20

Answer after your presentation, about **your own** program. Hand in this whole sheet.

5.1 Point (5 pts): Name your List, Set, and Map: the class you used for each, and what it holds.

5.2 Evaluate (5 pts): Pick one of your three. Why that collection and not the other two? Would choosing Hash instead of Tree, or Tree instead of Hash, make it better or worse?

5.3 Apply (5 pts): A school library app needs: (a) books found by call number, (b) learners waiting in line for one popular book, (c) the LRNs of everyone who borrowed this month, counted once each. Name the best collection class for each, and why.

5.4 Reflect (5 pts): Compare your program last week, with arrays, and this week, with collections. What can it do now that it could not do before?

✂ CUT ON THURSDAY · DETACH THE THURSDAY STRIP BELOW THIS LINE ✂

THURSDAY · OCT 22 Iteration: for loops and Iterator

Name: _____
Section: _____ Score: ____ / 15

Answer after Step 10 on page 5.

4.1 Define (5 pts): Name the three ways to loop through a collection, and one job each one is best for.

4.2 Explain (5 pts): Step 8 crashed with a `ConcurrentModificationException`, but Step 9 removed the same vehicles safely. What was different?

4.3 Connect (5 pts): Why can you not use an index `for` loop on a TreeSet or loop over a Map directly? What do you use instead for each?

Week 14 practice tasks

MONDAY - Beyond Arrays: The Java Collections Framework and the List Interface

Click Work in code editor to load this step's sample and supporting classes. Run the unchanged sample first. Sample verified saves that check and opens editing for Your task; it does not complete the step. Make the small alteration, then Run code again. Matching Expected task output marks Completed, saves progress, and unlocks the next activity. New steps load their own sample; reopening a started step restores saved edits. Earlier work and personal performance-task submissions stay separate.

Step 1 sample update: the loaded sample uses the stable line System.out.println("No vehicle there: index 5 is unavailable."); in its catch block. Its sample output ends with No vehicle there: index 5 is unavailable. This replaces the platform-specific exception message printed on the original page. Your task still changes it to No vehicle at index 5.

Step 1 Your task

In the sample, change fleet.remove(1) to fleet.remove(3). Remove the fourth vehicle instead of the tricycle. In the catch block, replace the printed exception message with exactly: No vehicle at index 5. Keep the list and the other output lines.

Expected task output

```
Vehicles in fleet: 4
First vehicle: NXA 123
After removal: 3
NXA 123 | Driver: Mang Ramon | Passengers: 0/20
TRC 456 | Driver: Kuya Ben | Passengers: 0/4
BUS 789 | Driver: Aling Rosa | Passengers: 0/50
No vehicle at index 5.
```

Step 2 Your task

Change the priority passenger from Lolo Pedro to Lola Nena. Keep addFirst, removeFirst, and the other three passengers unchanged.

Expected task output

```
Waiting: [Lola Nena, Ana, Ben, Carla]
Next to board: Lola Nena
Now boarding: Lola Nena
Still waiting: [Ana, Ben, Carla] (3 people)
```

TUESDAY - No Duplicates Allowed: The Set Interface with HashSet and TreeSet

Run the loaded sample first, then apply Your task below. Reopening this step restores your saved edits.

Step 3 Your task

Change only the third registered.add call: add BUS 789 instead of repeating NXA 123. Keep the BUS lookup and the removal of TRC 456.

Expected task output

```
true
true
true
Registered plates: 3
Is BUS 789 registered? true
After removing TRC 456: 2
```

Step 4 Your task

Keep hashStops as a HashSet. In the last println, display new TreeSet<String>(hashStops) instead of hashStops. This sorts only the displayed copy, giving everyone the same output. The sample HashSet order may differ between computers.

Expected task output

```
Stops passed: 7
Different stops: 5
TreeSet, sorted: [5th Avenue, Blumentritt, Divisoria, Monumento, Recto]
First alphabetically: 5th Avenue
Last alphabetically: Recto
HashSet: [5th Avenue, Blumentritt, Divisoria, Monumento, Recto]
```

WEDNESDAY - Find It by Its Key: The Map Interface with HashMap and TreeMap

Run the loaded sample first, then apply Your task below. Reopening this step restores your saved edits.

Step 5 Your task

In the try block, change the first findVehicle lookup from TRC 456 to NXA 123. Keep the later ZZZ 000 lookup so the custom exception is still handled.

Expected task output

```
Vehicles on record: 3
BUS 789 | Driver: Aling Rosa | Passengers: 0/50
Has ZZZ 000? false
get("ZZZ 000") gives: null
NXA 123 | Driver: Mang Ramon | Passengers: 0/20
Lookup failed: No vehicle with plate ZZZ 000
```

Step 6 Your task

Increase Kuya Ben's extra trips by 2 instead of 1. Change the + 1 in the update for Kuya Ben only. Keep the initial values and Mang Tonyo's update.

Expected task output

```
Drivers: 4
Aling Rosa: 3 trip(s)
Kuya Ben: 10 trip(s)
Mang Ramon: 5 trip(s)
Mang Tonyo: 1 trip(s)
```

THURSDAY - One at a Time: Mastering for Loops and the Iterator

Run the loaded sample first, then apply Your task below. Reopening this step restores your saved edits.

Step 7 Your task

Calculate the total fare for 6 km instead of 5 km. Change quoteFare(5) to quoteFare(6) and change the output label to 6 km. Keep the three Vehicle objects and their fare methods.

Expected task output

```
1. NXA 123
2. TRC 456
3. BUS 789
Total fare for 6 km on every vehicle: P127.0
```

Step 8 Your task

First run the unchanged sample. Expected error observed confirms java.util.ConcurrentModificationException; JVM stack-trace line numbers may differ. This verifies the demonstration only. The sample deliberately crashes when it removes from the list during for-each. For this task, list the empty vehicles without removing them. Replace fleet.remove(v); with System.out.println("Empty vehicle: " + v.getPlateNumber());. Keep the final fleet-size line. The next step teaches safe removal.

Expected task output

```
Empty vehicle: NXA 123
Empty vehicle: BUS 789
Vehicles still on duty: 3
```

Step 9 Your task

Use the Iterator sample. Change its condition from passengers == 0 to passengers > 0, so only the occupied tricycle goes off duty. Keep it.remove() and both output messages.

Expected task output

```
Off duty: TRC 456
Vehicles still on duty: 2
```

Step 10 Your task

Change Mang Ramon's trip count from 5 to 7. Keep the sorted stop list and all three map loops. Check both the total and Mang Ramon's final entry.

Expected task output

```
Stop: Divisoria
Stop: Monumento
Stop: Recto
Aling Rosa Kuya Ben Mang Ramon
Total trips: 19
Aling Rosa -> 3
Kuya Ben -> 9
Mang Ramon -> 7
```

FRIDAY - Data That Works: Collections Performance Task and Presentation

Run the loaded sample first, then apply Your task below. Reopening this step restores your saved edits.

Step 11 Your task

This is module practice using the worked store example, separate from your own performance-task project. Change Bread stock from 10 to 12. Keep the sales log, customers, sold-out removal, and both product lookups.

Expected task output

```
Ana added? true
Ben added? true
Ana added again? false
1. Ana bought 2 Bread
2. Ben bought 1 Softdrink
3. Ana bought 1 Softdrink
Pieces in stock: 86
Different customers: [Ana, Ben]
Sold out, removed: Candy
Bread -> 12
Rice (kg) -> 50
Softdrink -> 24
Bread in stock: 12
Search finished.
Sorry: No product named Eggs
Search finished.
```